

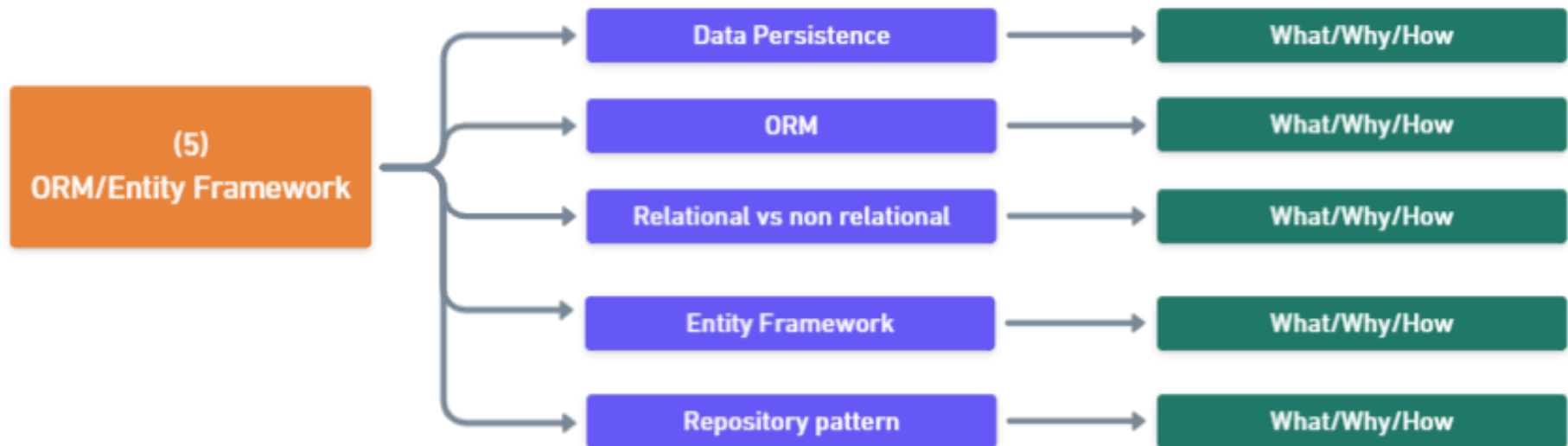
ORM / ENTITY FRAMEWORK

Tehnici avansate de programare

Nicoleta Scrimint



Agenda



Made with Whimsical

Data Persistence

Intro

- **Data persistence** refers to the mechanism by which data is **stored** and **retrieved**.
- Traditional methods include **file systems** and **relational databases**.
- Persistence ensures that **data remains available** even after the application that created it has terminated.



Data Persistence

Importance



✓ Data persistence is crucial for applications to **maintain state** and **functionality** across sessions.



✓ It enables data to be **stored**, **retrieved**, and **manipulated** over time.



✓ Persistence is essential for **scalability**, **reliability**, and **data integrity**.

Database operations in .Net

Intro

- Microsoft.Data.SqlClient package
- Example to query and insert data



- **ORM(Object-Relational Mapping)** is a programming technique that maps objects from an object-oriented domain to a relational database.



- It eliminates the need for developers to write complex SQL queries.
- ORM frameworks provide abstraction layers that simplify database interactions.

ORM

Benefits



- ✓ Increases productivity by **reducing** the amount of **code needed** for database operations.



- ✓ Improves maintainability by **encapsulating database logic** within application code.



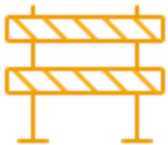
- ✓ Enhances portability by allowing developers to **switch between different database systems** without changing application code.



- **Performance overhead:** ORM frameworks may introduce additional overhead compared to handcrafted SQL queries.



- **Learning curve:** Developers need to learn the ORM framework conventions and best practices.



- **Limited control:** ORM may abstract away low-level database optimizations, limiting developers control over query execution.

Entity Framework Core ORM

Intro

- Example to query and insert data

Relational vs Non-Relational

Intro

- **Relational** databases **organize data** into **tables** with predefined schemas and **support complex queries** using SQL.



- **Non-relational** databases, such as NoSQL databases, **store data** in flexible formats like **key-value pairs**, **documents**, or **graphs**.



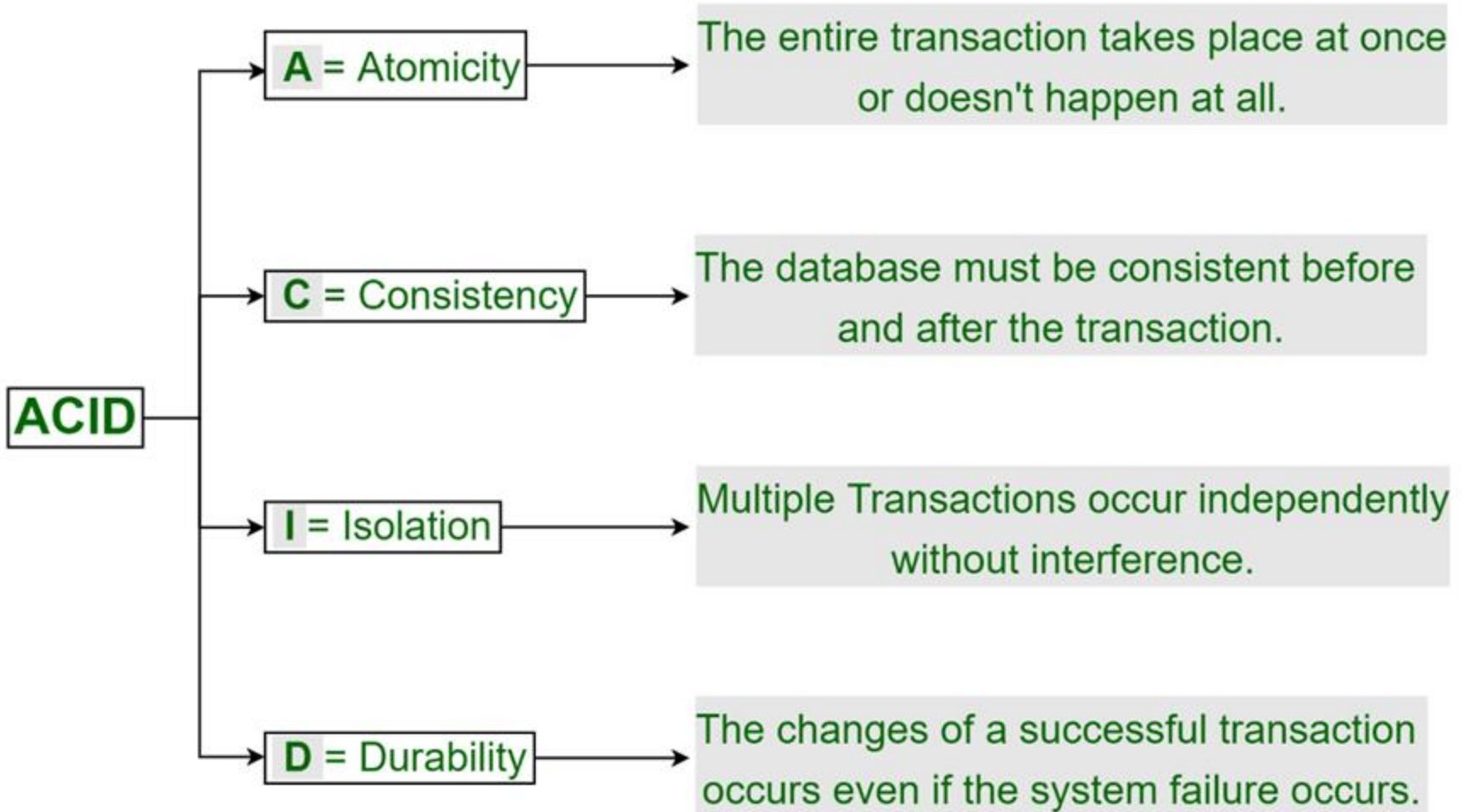
Relational Databases

Intro

- Relational databases organize data into **structured tables**, with each table consisting of rows and columns.
- They enforce data integrity through the implementation of **primary keys**, **foreign keys**, and **constraints**.
- Relational databases ensure data integrity through **ACID** transactions

Atomicity **C**onsistency **I**solation **D**urability

Relational Databases



Relational Databases

Examples:



Non-Relational Databases

- Non-relational databases, also known as NoSQL databases, offer **flexible data models** suitable for **various data types** and **structures**.
- They do not adhere to the traditional tabular structure and allow for **storing unstructured**, **semi-structured**, and **polymorphic data**.
- NoSQL databases provide **horizontal scalability**, making them suitable for handling **large volumes** of data and **distributed systems**.

Non-Relational Databases

Examples:

- document-oriented



- key-value stores



- graph databases



Relational vs Non-Relational

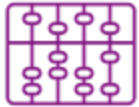
Choosing

Choosing between **relational** and **non-relational databases** depends on factors such as:

- Data structure complexity
- Scalability requirements
- Performance needs

Relational vs Non-Relational

Choosing



- **Data structure complexity:** **Relational databases** are ideal for structured data with well-defined schemas, while non-relational databases offer flexibility for unstructured or semi-structured data.



- **Scalability requirements:** **Non-relational databases** excel at horizontal scalability, making them suitable for distributed systems and big data applications.



- **Performance needs:** **Relational databases** are optimized for complex queries and transactional consistency, while NoSQL databases prioritize high volume for read and write operations.

Relational vs Non-Relational

Choosing

- Development and maintenance:
 - **Relational databases** often require predefined schemas and normalized data
 - **NoSQL databases** offer schema flexibility and agile development approaches.



It's essential to evaluate these factors based on the specific requirements of your application and consider factors such as **data access patterns**, **deployment architecture**, and **team expertise**.

Entity Framework

Intro

- **Entity Framework** (EF) is a popular ORM framework for .NET applications.
- It enables developers to work with **relational data using domain-specific objects**.
- **EF supports** various database providers, including:
 - SQL Server
 - MySQL
 - SQLite
 - PostgreSQL
 - Oracle

Entity Framework

Components

- Entity Framework consists of three main components:
 - **Entity Data Model**
 - **Object Services**
 - **Entity SQL**

Entity Framework

Components



- **Entity Data Model** defines the conceptual model of the application's data.



- **Object Services** provide APIs for querying, inserting, updating, and deleting objects.



- **Entity SQL** allows developers to query entities using a syntax like SQL.

Entity Framework - examples

Components

- Creating and Configuring a Model
- Querying Data
- SQL Queries
- Saving Data

Entity Framework

Key Features



- **Automatic code generation:** EF can generate entity classes from an existing database schema (and schema from classes).



- **LINQ integration:** EF seamlessly integrates with Language Integrated Query (LINQ) for querying data.



- **Change tracking:** EF tracks changes to entity objects and generates appropriate SQL statements for database updates.

Entity Framework

Migrations

- EF Migrations enable developers to **evolve** the database schema **alongside application** changes.
- Migrations **automatically generate SQL scripts** to update the database schema based on changes to the Entity Data Model.
- This ensures database **schema consistency** across different **environments**.

Entity Framework – migrations example

- Migrations Overview

Entity Framework

Let's see the three approaches as follows:



- **Database first** – Database first is the reverse engineer approach where the existing database generates the data model, and which maps it entirely.



- **Code first** – Code first approach defines the data model in the application code and then finally uses the code for generating the new database.



- **Model first** – The model first approach generates the database using the data model created with the EF Designer.

Entity Framework

Performance

Lazy loading vs. eager loading

- Developers can optimize performance by **controlling** when related entities are **loaded into memory**

Query performance

- EF offers various querying techniques, such as **compiled queries** and **raw SQL** execution, to improve performance

Caching

- EF supports **caching mechanisms** to reduce database round trips and improve performance

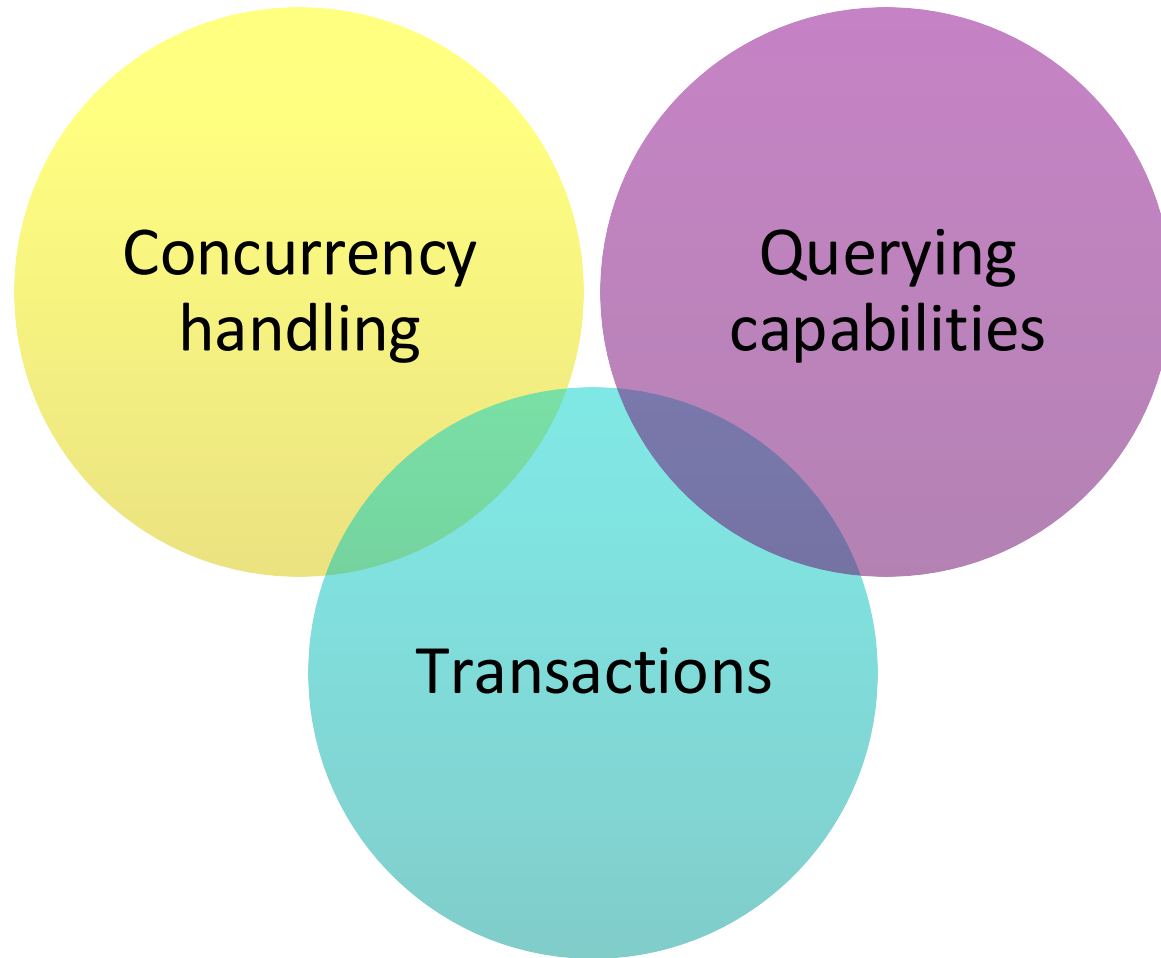
Entity Framework

Performance

- Loading Related Data
- Advanced Performance Topics

Entity Framework

Advanced



Entity Framework

Advanced

- **Concurrency handling:** Entity Framework provides mechanisms to handle concurrent data modifications, ensuring data consistency in multi-user environments.
 - **Optimistic** concurrency control allows multiple users to access and modify data simultaneously, relying on versioning or timestamping to detect conflicts.
 - **Pessimistic** concurrency control locks data resources to prevent concurrent access, ensuring that only one user can modify a record at a time.
- **Transactions:** Entity Framework supports transactions to maintain data integrity and consistency across multiple database operations.
- **Querying capabilities:**
 - Entity Framework provides powerful querying capabilities using LINQ (Language Integrated Query) or Entity SQL.

Entity Framework

Advanced

- Handling Concurrency Conflicts
- Using Transactions
- Complex Query Operators
- Pagination
- Database Functions
- User-defined function mapping

Repository pattern

Intro



- The **Repository pattern** is a design pattern that **separates** the **data access logic** from the **business logic** of an application.



- **Repositories encapsulate** data access **operations** and provide a layer of abstraction over the underlying data source.

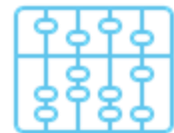


- This pattern promotes **modularity**, **testability**, and **maintainability** in applications.

Repository pattern

Benefits

- **Decouples data access logic** from the rest of the application, promoting code reusability and maintainability.
- **Facilitates unit testing** by allowing data access operations to be mocked or stubbed.
- **Provides a consistent interface** for interacting with different data sources, such as databases, web services, or in-memory collections.



Repository pattern

Implementing

- In Entity Framework, **repositories** serve as an **abstraction layer** over **DbContext**, which represents the database context.
- Repositories encapsulate **CRUD** (Create, Read, Update, Delete) operations for entity types.
- This approach **enhances code organization** and promotes **separation of concerns** in applications.

Repository pattern

Implementing

- Implement the infrastructure persistence layer with Entity Framework Core

Summary

Data Persistence



- The ability to persist data is fundamental to application development, ensuring that data remains available and consistent across sessions.

ORM (Object-Relational Mapping)



- ORM simplifies database interactions by abstracting away the complexities of SQL queries and mapping database tables to domain-specific objects.

Relational vs. Non-Relational Databases



- The choice between relational and non-relational databases depends on factors such as data structure complexity, scalability requirements, and performance needs.

Summary

Entity Framework



- Entity Framework provides a powerful ORM framework for .NET applications, offering features such as automatic code generation, LINQ integration, and support for database migrations.

Repository Pattern



- The Repository pattern promotes modularity, testability, and maintainability in applications by separating data access logic from the rest of the application.